JEP 540 · JDK 28 · INCUBATOR

Everyday JSON, built into Java

Reading and writing JSON with nothing but the JDK — what it is, what it changes for us, and a live demo.

Based on the Devoxx talk “Everyday JSON” by Viktor Klang & Alan Bateman (Oracle Java Platform Group)

WHAT JAVA DEVELOPERS SAY

Java developers have been asking for this

“Too many dependencies for a simple job”

I maintain my own JSON helper and pull in Gson just to parse.

A basic JVM web service needs an HTTP library *and* a JSON library.

The small Java JSON library I rely on is no longer maintained.

“It should simply be standard”

I've used an external JSON standard and its implementations for years. Why isn't it in the JDK?

I want a common, well-maintained JSON API without the complexity of JSON-B.

“Let me explore any JSON, safely”

After 20 years of Jackson, I want that easy tree navigation built in.

Will numbers stay exact, and what about duplicate keys? The JEP keeps exact values and rejects duplicates.

Paraphrased from Java developers in the Hacker News discussion of JEP 540 (July 2026): 102 points, 82 comments

THE PROBLEM

JSON is everywhere. Java is the odd one out.

Python	Go	JavaScript	C#	Java ≤ 27
`import json`	`encoding/json`	`JSON.parse`	`System.Text.Json`	`add a library`
Built in	Built in	Built in	Built in	Jackson, Gson…

Today, even a 10-line script that reads a config file or calls a REST API needs a build tool and a third-party dependency.

RECAP · WHAT IS A JEP?

A JEP is how a feature gets into Java

JDK Enhancement Proposal — a public design document from OpenJDK. Every major Java feature (records, virtual threads, pattern matching) started as one.

STAGE 1 · WE ARE HERE

Incubator

Experimental module. Opt-in flag. API may change or be dropped.



STAGE 2

Preview

Fully specified, still needs --enable-preview. Last call for feedback.



STAGE 3

Final

Standard Java. Stable forever. Safe for production.

JEP 540 · SIMPLE JSON API

Simple things, done simply

What it is

- A JSON parser & writer inside the JDK
- Module jdk.incubator.json
- Strict RFC 8259 JSON
- Immutable, type-safe tree of values
- Integrated in JDK 28 (March 2027)

What it is not

- Not a Jackson replacement
- No object mapping (POJO ↔ JSON)
- No streaming of huge files
- No comments, no trailing commas
- Not production-ready yet

FOR THE BUSINESS

Why should non-developers care?

Fewer dependencies

Less third-party code to audit, patch and license-check.

Faster small wins

Scripts, test tools and prototypes in minutes, not a project setup.

Better ops tooling

The JDK already exports thread dumps and JFR data as JSON. Now Java can read them too.

AI-friendly

A simple standard API is easier for AI assistants to generate correctly.

THE API IN 60 SECONDS

Six types, two directions

sealed JsonValue

- JsonObject
- JsonArray
- JsonString
- JsonNumber
- JsonBoolean
- JsonNull

Immutable tree · works with switch pattern matching

```
// text → tree
Json.parse(text)

// navigate
.get("name") .get(0)
.tryGet("port") → Optional

// convert (no guessing)
.asString() .asInt() .asList()

// tree → text
value.toString() // compact
Json.toDisplayString(v, " ")
```

Read a config file, port is optional

BEFORE · Java 21/25 + Jackson

```
// + pom.xml dependency, + jars
var mapper = new ObjectMapper();
JsonNode json = mapper.readTree(
  Path.of("config.json").toFile());
var cs = Charset.forName(
  json.get("charset").asText());
int port = json.has("port")
  ? json.get("port").asInt() : 8080;
```

NOW · JDK 28, nothing to add

```
// only the JDK
var json = Json.parse(
  Files.readString(Path.of("config.json")));
var cs = Charset.forName(
  json.get("charset").asString());
int port = json.tryGet("port")
  .map(JsonValue::asInt).orElse(8080);
```

Same length — but zero dependencies, and **"port": "abc"** fails loudly instead of Jackson's silent **asInt() → 0**.

Send a JSON body with the built-in HttpClient

BEFORE · string building (or a library)

```
// escaping is your problem
String body = """
  {"user": "%s",
   "msg": "%s"}
  """.formatted(user, msg);

// msg = He said "hi"
// → broken JSON sent
```

NOW · a real JSON value

```
// escaping handled for you
var body = JsonObject.of(Map.of(
  "user", JsonString.of(user),
  "msg",  JsonString.of(msg)));

HttpRequest.newBuilder(uri)
  .POST(BodyPublishers
    .ofString(body.toString()))
```

HttpClient + JSON, both in the JDK: a working REST client with zero dependencies.

A one-off question: average temperature this week

```
// Weather.java — run it directly, no build
var json = Json.parse(response.body());
json.get("properties").get("periods")
    .asList().stream()
    .mapToInt(p → p.get("temperature").asInt())
    .average()
    .ifPresent(IO::println);

$ java --add-modules jdk.incubator.json Weather.java
```

BEFORE

New Maven project, add Jackson, build a jar — or give up and use Python.

NOW

One .java file, a few lines, run it. Works in JShell too.

DESIGN CHOICES

Strict by design: the same JSON means the same thing

RFC 8259 only

No comments, trailing commas or duplicate keys.

No silent coercion

asString() on a number throws. No guessing.

Fail fast, with the path

Errors name the missing key, the path and the offset.

```
// old ids were "42", new ones 42
long id = switch (v) {
  case JsonNumber n →
    n.asLong();
  case JsonString s →
    Long.parseLong(s.asString());
  default → throw new
    IllegalArgumentException("bad id");
};

// sealed types = modern Java
```

WHAT IT MEANS FOR US

We run Java 21 and 25. What changes?

	Java 21 LTS	Java 25 LTS	JDK 28 · Mar 2027	JDK 29 LTS · Sep 2027
JSON in the JDK?	No	No	Yes, opt-in flag	Depends on progress
Production use	Keep Jackson	Keep Jackson	Not yet	Only once Final
Our existing code	Unchanged	Unchanged	Unchanged	Unchanged
What we do	Nothing	Nothing	Try it, give feedback	Re-evaluate

Zero risk today: nothing breaks, nothing to migrate. The opportunity is to **shape the API now**, while it is still an incubator.

PICK THE RIGHT TOOL

Start simple, graduate when you need more

JDK JSON API

- Scripts, CLI tools, JShell
- Test assertions on JSON responses
- Small config files
- Reading thread dumps / JFR output
- Prototypes and POCs

Jackson / Gson

- REST APIs (Spring, Quarkus…)
- Object mapping to POJOs / records
- Streaming very large files
- Lenient formats (comments…)
- Anything in production today

LIVE DEMO

Let’s run it.

One JDK. No Maven. No Jackson. Three small programs.

DEMO · SCRIPT

Three steps, five minutes

1

Read a config file — then break it

```
java --add-modules jdk.incubator.json ConfigDemo.java
```

Set "port": "abc" and rerun: a clear error, not a silent 0.

2

Explore JSON live in JShell

```
jshell --add-modules jdk.incubator.json
```

Parse, navigate, pretty-print with toDisplayString.

3

Analyse a thread dump of our own app

```
jcmd <pid> Thread.dump_to_file -format=json dump.json
```

List virtual threads with a 20-line tool. Next slide.

DEMO · THE OPS WIN

Which virtual threads are running in prod?

```
// ThreadDumpDemo.java
var dump = Json.parse(Files.readString(path));
for (var c : dump.get("threadDump")
    .get("threadContainers").asList()) {
  c.get("threads").asList().stream()
   .filter(t → t.tryGet("virtual")
     .map(JsonValue::asBoolean)
     .orElse(false))
   .forEach(t → IO.println(
     t.get("name").asString()));
}
```

Input

JSON thread dump the JDK already produces (Java 21+).

Output

A custom diagnostic tool in one file. No project, no jars.

OUR PROPOSAL

Small bet, zero production risk

NOW

Try it

Install the JDK 28 build on dev machines. Use it for one internal tool: thread-dump or log analysis.

NEXT 3 MONTHS

Give feedback

Send our real use cases to core-libs-dev@openjdk.org while the API can still change.

AT NEXT LTS

Decide

When it is Final, reassess for tests, tooling and small services.

Ask: **[1–2 volunteers]** and **[__ days]** of exploration time.

TL;DR

A JSON API for when you don’t have anything else.

Built into JDK 28, opt-in incubator

Nothing changes for Java 21 / 25

Try it now, shape it, adopt it when Final

Questions? · openjdk.org/jeps/540